Recursive Block-Sparse Language Models

SR-Core: A Hard Working-Set Guarantee for Cache-Efficient Parameter Streaming — A Research Monograph

Viktor Jedich

2026

Contents

License and Attribution

Recursive Block-Sparse Language Models — Research Monograph

Citation:

```
@techreport{jedich2026srcore,
  title   = {Entropy-based Router Consolidation for Cache-Efficient
             Recursive Block-Sparse Language Models},
  author  = {Jedich, Viktor},
  year    = {2026},
  note    = {Self-published research monograph.
             \url{https://github.com/vijedich/sr-core-recursive-blocklm}}
}
```

Status: Self-published research monograph / technical report. Not peer-reviewed, not a doctoral dissertation.

Chapter 1: Motivation and Problem Statement

How to Read This Monograph

This document is a self-published research monograph, not a conference paper and not a thesis. It records a small-scale but end-to-end investigation of one architectural question:

> *Can a language model be structured so that, for each token, only a small and predictable subset of its parameters must be resident on the GPU?*

The work was conducted under consumer-hardware constraints — primarily an RTX 2060 (6 GB VRAM). Its purpose is not to claim deployment-scale performance, dense-quality parity, or a production-ready system. Instead, it documents what could be built, trained, measured, falsified, and bounded within those constraints.

What was simulated vs. what was measured:

Claim	Status
WS=k guarantee (architectural)	Proved and empirically verified (Ch. 2–3)
4.1× transfer reduction vs. dense	LRU simulation on real trained models (Ch. 5b.0)
Real wall-clock throughput advantage	Measured on RTX 2060 under forced offloading (Ch. 5b.4)
~0.5 nats quality gap behind Dense d24	Measured, param- and compute-matched (Ch. 5a.5)
Advantage at deployment scale	Not tested — requires larger models and VRAM budget
Quality gap at convergence	Not tested — both models undertrained at 15k steps

The story arc in one paragraph:

The initial question was whether fewer bytes in motion would mean more tokens per second. Simulation said yes. The first naive RAM→VRAM prototype said no — per-block kernel launches dominated and made SR-Core slower than dense. Fusing those launches into a grouped block matmul revealed the structural advantage: 297 vs. 205 tok/s. Asynchronous two-stream overlap added another 5–9% in the transfer-limited regime. The mechanism is real and measured on this hardware. The open question is whether it survives at deployment scale — larger blocks, models that genuinely exceed VRAM, and batched serving. That test requires hardware beyond what is available here.

Open questions are structured as continuation points in Section 7.5 — not as limitations to apologize for, but as discrete, well-scoped experiments for anyone with the right hardware or the right curiosity to pick up.

1.1 The Inference Bottleneck on Consumer Hardware

Large language models with 30B, 70B, or 100B+ parameters exceed the VRAM capacity of typical consumer and prosumer GPUs. The dominant solution in practice is *layer-by-layer offloading*: weights reside in CPU RAM or on SSD, and each layer is transferred to VRAM before computation, then evicted afterward.

On a typical consumer GPU with 6–8 GB VRAM and PCIe bandwidth of ~16 GB/s:

```
7B model in fp16  ≈  14 GB weights
Per-token cost    =  14 GB ÷ 16 GB/s  ≈  0.9 seconds
→ approximately 1 token/second
```

This is not a software optimization problem. It is a structural problem: a densely executed model requires *all* its weights at every token, so all of them must be transferred.

1.2 The Core Hypothesis

A language model can be trained such that for each token, only a small, predictable fraction of its weights is needed.

If this active fraction is:

- **small** (1–5% of total weights per token),
- **predictable** (the next required block is known before it is needed),
- **reproducible** (similar inputs activate similar weights),

then a model whose total size exceeds available VRAM can run on small hardware — not because it is faster, but because it never needs the full model simultaneously.

The goal is enablement, not optimization. The baseline is layer-by-layer offloading, not dense in-VRAM execution.

1.3 Relationship to Existing Approaches

Approach	Mechanism	Limitation
Layer offloading (llama.cpp)	Load each layer sequentially	100% of weights per toke
Quantization (GPTQ, AWQ)	Compress weights	Fewer bytes, but still all
Standard MoE (Mixtral)	Top-k of N experts per layer	Per-layer, no cross-step p
This work	Shared block bank, fixed active set, fixed routing	Predictable WS=k indepe

Standard MoE reduces the *number* of active parameters per layer but does not bound the active set across recursion steps or enable routing reuse. Each layer still selects independently.

1.4 Research Questions

This monograph investigates four questions:

1. **Feasibility:** Can a block-sparse recursive model learn language modeling tasks at all, and does its routing remain stable (no collapse to a few blocks)?
2. **Working-set guarantee:** Does fixing routing at the first recursion step and reusing it for subsequent steps (SR-Core) preserve language quality while guaranteeing WS=k independent of bank size n and recursion depth R?
3. **Cache efficiency:** Can router consolidation (entropy-based regularization) improve the predictability and locality of active block sets without degrading language quality?

4. **Offloading relevance:** How does the simulated bytes-in-motion of SR-Core compare to dense layer-offloading, and what is the effect of consolidation on this metric?

1.5 Scope and Limitations

This work operates at small scale: a ~19M parameter model trained on a ~6.6M token four-domain corpus. Results are not transferable to large-scale models without further validation. A small-scale RAM→VRAM prototype demonstrates a measured throughput advantage under forced offloading (Chapter 5b.4); deployment-scale throughput, batched serving, and large-block scaling remain untested. The CPU-side dispatch overhead (~2.8× relative to compute-matched dense, Chapter 4) limits throughput in the compute-bound regime; it becomes sub-dominant only in the transfer-bound regime after sparse dispatch has been fused to remove per-block kernel-launch overhead.

1.6 Chapter Overview

- **Chapter 2** formalizes SR-Core and proves the WS=k guarantee.
- **Chapter 3** validates the guarantee empirically across bank sizes n ∈ {16, 32, 64, 128, 256} on TinyStories.
- **Chapter 4** measures the dispatch overhead of sparse block selection.
- **Chapter 5** evaluates SR-Core on HeteroMini-v1, a four-domain corpus, including offloading simulation and cross-seed robustness.
- **Chapter 6** introduces entropy-based router consolidation and characterizes its effect on cache efficiency and language quality.
- **Chapter 7** synthesizes the findings and outlines directions for future work.

Chapter 2: SR-Core Architecture and the Working-Set Guarantee

2.1 Background: Free-Routing Variants

Before SR-Core, earlier variants of the block-sparse recursive model selected blocks *independently* at each recursion step r. Formally, each step maintained its own routing decision:

```
S_r = top-k(Router(h_r))    for each r = 1, 2, …, R independently
```

Analysis of these free-routing variants revealed a **two-phase routing structure**:

- At r=1, routing is token-specific (mean inter-token Jaccard $J_1 \approx 0.20$): the first step broadly explores the block bank.
- At r=2…R, routing collapses to near-identical selections across tokens ($J_{2–6} \approx 0.984$): the deeper steps overwhelmingly reuse the same blocks.

This empirical observation motivates a key design question: if steps r=2…R almost always select the same blocks as r=1, why compute a separate routing decision for each?

Additionally, a forced-diversity experiment (entropy bonus at r=6, the deepest step) reduced language modeling loss by 0.104 nats, indicating that routing collapse at deep steps is a **training artifact** — the model learns to avoid the diversity penalty but would benefit from more novel block selections at depth.

2.2 SR-Core: Shared Routing with Fixed Active Set

SR-Core replaces per-step routing with a single routing decision at r=1, reused for all subsequent steps.

2.2.1 Block Bank

A shared bank of n parameter blocks:

```
B = {W₁, W₂, …, Wₙ}
```

Each block implements a feedforward transformation:

```
F_b(h) = W₂,b · σ(W₁,b · LayerNorm(h))
```

2.2.2 Router

At recursion step r=1, a noisy top-k router selects k blocks:

```
logits_b = q · k_b + ε_b,     ε_b ~ Gumbel(0, 1)
S = top-k({logits_b : b ∈ B}, k)
```

where q = Wq · h_1 is the routing query and k_b is the learned key for block b.

2.2.3 Reuse Rule (SR-Core)

The selection S from r=1 is reused unchanged for r=2, 3, …, R:

```
S_r = S₁    for all r = 1, …, R
```

2.2.4 State Update

At each step r, the state is updated by the selected blocks:

```
h_{r+1} = h_r + Σ_{b ∈ S} α_{r,b} · F_b(h_r)
```

where α_{r,b} are normalized gates over the selected k blocks.

2.2.5 Deep-Supervision Output

A readout head is applied after each step:

```
p_r(x_{t+1}) = softmax(W_out · h_r)
L = Σ_r w_r · CrossEntropy(p_r, x_{t+1})
```

where w_r are step weights (end-heavy weighting is used to preserve the anytime property without flattening the depth curve).

2.3 The Working-Set Guarantee

Theorem (WS = k). Under the SR-Core reuse rule, the working set — the number of distinct blocks accessed during the processing of a single token across all R recursion steps — equals exactly k, independent of n and R.

Proof sketch. Since S_r = S_1 for all r, the union of active blocks is:

```
Union(S_1, S_2, …, S_R) = S_1
|S_1| = k
```

regardless of R or n. □

This is the fundamental architectural property that enables predictable weight caching: the complete set of weights needed for a given token is known after the first routing decision and does not grow with recursion depth.

Comparison with Layer-Offloading

Architecture	Bytes loaded per token	Grows with depth?
Dense layer offloading	All weights	Yes (one pass per layer)
Standard MoE (per-layer)	k × block_size × num_layers	Yes (independent per layer)
SR-Core	**k × block_size**	**No**

2.4 Why Route Only at r=1?

The empirical motivation from free-routing analysis ($J_{2–6} \approx 0.984$) shows that routing decisions at deep steps are nearly identical across tokens. Fixing them to r=1 trades marginal routing adaptability (at steps where it barely exists) for a hard WS guarantee.

The pre-topk application of the entropy objective (described in Chapter 6) operates on the routing distribution before the discrete selection, preserving the differentiable path while controlling the concentration of the pre-selection distribution.

2.5 Implementation Details

Dispatch: True sparse dispatch — each token is processed only by its k selected blocks, not by all n blocks with masking. This is essential for the WS guarantee to translate into actual memory footprint reduction.

Load balancing: A soft load-balancing auxiliary loss encourages uniform block utilization across the training batch. Without it, a small subset of blocks dominates all routing decisions (collapse).

Model configuration (this work): n=64 blocks, k=8, R=6, d=256, block hidden dimension 512, ~19M parameters total.

2.6 Relationship to Prior Work

SR-Core shares structural similarities with Universal Transformers and Deep Equilibrium Models (shared weights applied repeatedly) but differs in two key properties:

1. Routing is content-dependent (different tokens activate different blocks)
2. The active set is fixed at r=1 (static across recursion depth)

Neither Universal Transformers nor DEQ address the predictability of the active weight set at inference time, which is the property required for efficient offloading.

SR-Core also differs from standard MoE in that the routing is shared across depth: in a 6-layer MoE, a token loads k experts at each of 6 layers = 6k distinct expert loads. In SR-Core with R=6, a token loads exactly k blocks total.

A full literature review — covering MoE load-balancing, looped depth (Universal Transformer, DEQ), parameter offloading (FlexGen), and routing regularization — is in Chapter 8.

Chapter 3: Scaling Validation — WS Independent of Bank Size

3.1 Research Question

The working-set guarantee WS=k is an architectural property (Chapter 2). This chapter asks whether it holds *empirically* at the language quality level: does training with larger block banks (larger n) degrade quality relative to the k=const active budget, and does the sparse routing continue to match dense quality at matched compute?

3.2 Experimental Design

3.2.1 Corpus

TinyStories (Eldan & Li, 2023): synthetic short stories, simple vocabulary, ~2M tokens. Used for Phase-1 scaling because it is small enough for CPU-scale experiments and provides a clear language modeling signal without domain complexity.

3.2.2 Model Configurations

All models use k=4 active blocks, R=6 recursion steps, d=128 (demo scale). Bank size is varied across: n ∈ {16, 32, 64, 128, 256}.

Baseline models:

- **Dense A (16 blocks):** All 16 blocks applied once each. Quality upper bound at this parameter count.
- **Recurrent B (1 block × 16):** Single shared block, applied 16 times. Isolates the effect of pure recursion without routing.
- **Routed C (4/n):** SR-Core with varying bank size n.

3.2.3 Training Protocol

3,000 steps, AdamW, cosine schedule, deep supervision (end-heavy weighting w_R=0.7). Cross-seed validation: s0, s1, s2 for key configurations.

3.3 Results: Phase-1 Baseline

At the initial synthetic-data scale (n=24 blocks, k=4, 16 block-applications, CPU demo config with d=128):

Model	Active blocks	L_1	L_final	Quality
Dense A (16 distinct)	16	0.607	**0.589**	Baseline
Recurrent B (1 × 16)	1	0.756	**0.720**	Worst
Routed C (4/24)	4	0.547	**0.547**	**Best**

Note: the n=24 demo config uses a smaller bank than the TinyStories scaling runs (n ∈ {16, 32, 64, 128, 256}, section 3.4) because it is the original CPU prototype config. The architecture is the same; the TinyStories experiments use the same codebase at larger bank sizes and on GPU.

At equal compute (16 block-applications each): **C < A < B**.

Key finding: routing C achieves better quality than dense A with only 4 active blocks (vs. 16 for A). The quality advantage comes from selective activation, not from additional compute.

Recurrent B degrades with depth (U-shaped curve, minimum ≈0.680 at step 5, then worsening): pure recursion without routing diversity is not beneficial and becomes unstable at depth.

3.3.1 Routing Health

- Router entropy: 0.984 (near-perfect balance; 1.0 = uniform)
- Dead blocks: 1 of 24 (4%)
- Maximum single-block utilization: 0.092 (no dominant block)
- MI_norm (regime ↔ block assignment): 0.197 ± 0.030 (stable specialization)

3.4 Bank Size Scaling

These experiments use *free-routing* variants (independent routing per step, pre-SR-Core) trained for 3,000 steps on TinyStories with k=4, R=6, d=128. They demonstrate that quality does not degrade — and modestly improves — as the block bank grows, while routing health is maintained.

Note: free-routing means the routing decision is made independently at each step r. The two-phase structure (J_{12} low, J≥23 high) emerges from training. In SR-Core, routing is fixed at r=1, so WS=k=4 exactly. Here, the empirical unique-blocks/token is higher (≈6–7) because step r=1 explores different blocks than steps r≥2.

Bank (n)	L_1	L_final	J(r1→r2)	Unique/tok	Dead	Router entropy
16	3.743	3.741	0.377	5.95	0	0.950
32	3.692	3.693	0.237	6.74	0	0.979
64	3.714	3.720	0.184	7.07	0	0.994
128	3.628	**3.630**	0.098	7.52	0	0.995
256	3.638	3.650	0.125	7.38	1	0.998

Key observations:

- **Quality improves with n** (b128 is best). A larger block vocabulary gives the router more specialized units to choose from without increasing the active count k.
- **J(r1→r2) decreases with n**: larger banks make step-1 routing more token-specific (more choices → lower overlap with the collapsed r≥2 selections). This is the two-phase structure becoming more pronounced.

- **Router entropy is near-uniform at all sizes** (0.950–0.998). No collapse at any bank size tested.
- **One dead block at n=256** — the only routing health anomaly; all other bank sizes have zero dead blocks.

Cross-seed validation (b32 and b64): L_final s1=3.814, s2=3.655 for b32; additional seeds show ≈±0.06 variance, consistent with the synthetic-task scale.

3.5 Anytime Property

With end-heavy loss weighting (w_R = 0.7), the per-step loss curve L(r) decreases with depth for difficult input patterns:

- REPEAT regime: L improves 1.210 → 1.178 with depth
- INCREMENT/ALTERNATE: solved at r=1 (≈0.05 / ≈0.001), marginal gain thereafter
- FIB (hardest): L ≈ 1.06, no depth gain — too hard for this scale

The anytime property is regime-dependent. Easy patterns converge at r=1; hard patterns benefit from depth. This motivates an adaptive halting mechanism (future work) that stops early for easy inputs.

Equal-weight supervision (all w_r = 1/R) produces a flat anytime curve: it strongly trains r=1, making additional depth redundant. End-heavy weighting increases the depth benefit for REPEAT by 10× compared to equal weighting.

3.6 Routing Specialization

3.6.1 Phase-1 Synthetic (REPEAT/FIB/INCREMENT/ALTERNATE Regimes)

Block activation correlates with input regime (measured by MI between regime label and active block set):

Seed	MI_norm	L_final
s0	0.213	0.550
s1	0.155	0.716
s2	0.223	0.623
Mean ± std	**0.197 ± 0.030**	

Ablation impact (removing blocks with highest per-regime activation preference):

- REPEAT-specific blocks: +0.077 on REPEAT loss
- FIB-specific blocks (shared with REPEAT): +0.162 on REPEAT, +0.044 on FIB
- INCREMENT/ALTERNATE blocks: no measurable effect (routing redundancy)

Specialization emerges where computation is actually needed. Trivially-solved regimes are multiply covered — ablating their blocks does not noticeably hurt quality.

3.6.2 TinyStories Linguistic Competence Analysis

Model: `competence_b64k4R6_s0` (n=64, k=4, R=6, TinyStories, 3,000 steps)

Six linguistic competence categories annotated by keyword detection:

Category	n samples	L_cat	Ablation Δ
scene_shift	9,265	3.807	+0.006 nats
causality	2,112	3.911	**+0.118 nats**
dialogue	835	4.039	**+0.081 nats**
emotion	1,120	4.092	+0.005 nats
coreference	24,058	4.177	+0.004 nats
temporal	934	4.233	+0.004 nats

(Ablation Δ = mean loss increase over R=6 steps when ablating blocks with highest activation rate for that category)

Routing classifier accuracy: 41.7% vs. chance 16.7% (6 classes). Bootstrap CI: [38.4%, 43.2%]. The routing signal carries significant categorical information at every recursion step (r=4 is best: 42.4%).

MI per recursion step:

- r=1: 0.0084 bits — exploration phase (low categorical signal)
- r=2–6: ~0.0103 bits — stable plateau, 23% above r=1

This mirrors the two-phase structure: at r=1 the router explores broadly; from r=2 onward it settles into a stable assignment carrying more categorical information.

Causality (+0.118) and dialogue (+0.081) have clear specialist blocks. Ablating them meaningfully degrades those categories. Coreference, emotion, scene_shift, and temporal show near-zero ablation impact (shared generalist blocks).

The hardest category (temporal, L=4.233) and the most frequent (coreference, n=24,058) both have near-zero ablation impact — routing specializes where dedicated computation provides a measurable quality gain, not simply for the hardest or the most common tasks.

Hub structure (b64, free-routing):

- Gini coefficient: 0.575 (moderate imbalance — hub blocks present)
- Top-5 block share: 40.1% of total usage across n=64
- Cache simulation: K=8 → 20.5% miss rate (learned) vs. 89.6% (random routing baseline)

3.6.3 Cross-Seed Variance of Categorical Specialization

clf_accuracy and MI across 4 seeds (s0 = base; s1–s3 = curriculum_fromIter2 variant):

Seed	clf_acc (all r)	Bootstrap mean	Bootstrap CI	MI r=1	MI r≥2
s0	41.7%	40.9%	[38.4%, 43.2%]	0.0084	0.0103
s1	38.9%	40.1%	[38.1%, 42.9%]	0.0046	0.0090
s2	36.3%	39.6%	[36.1%, 42.7%]	0.0053	0.0101
s3	38.5%	38.4%	[36.2%, 40.4%]	0.0051	0.0121
Mean ± std	**38.9% ± 2.0%**	**39.7%**		**0.0059**	**0.0104**

All four seeds are substantially above 16.7% chance, and all bootstrap CIs exclude chance. The routing classifier is reliably above chance across seeds.

The two-phase MI pattern holds in every seed: MI at r=1 is consistently lower than MI at r≥2, and the r=1 value varies more across seeds (0.0046–0.0084) than the plateau (0.0090–0.0121), consistent with r=1 being the less-constrained exploration step.

Cross-seed ablation (now computed for all seeds; data in `data/eval/phase1/competence_ablation_s{1,2,3}.json`)

Ablation-Diagonale: Δ_self − Δ_other (positive = category-specific specialist block)

Category	s0 Δ(self−other)	s1 Δ(self−other)	s2 Δ(self−other)	s3 Δ(self−other)	Consistent?
causality	**+0.114**	+2.11	+0.00	+1.89	3/4 seeds
dialogue	**+0.073**	−1.27	+1.05	+1.58	3/4 seeds
temporal	+0.000	+0.00	**+3.20**	+0.00	1/4 seeds
coreference	+0.000	+0.17	+1.04	+0.00	2/4 seeds (weak)
emotion	+0.000	−2.02	+0.00	+0.00	0/4 seeds
scene_shift	+0.000	+0.00	+0.00	−0.01	0/4 seeds

Key finding: Causality (3/4 seeds) and Dialogue (3/4 seeds) are the most consistently category-specific. This replicates the s0 result and holds across the curriculum variant. Emotion and scene_shift show no consistent specialist signal in any seed.

Scale difference between s0 and s1–s3: The absolute ablation deltas in s1–s3 are 10–30× larger (e.g., causality self-ablation: +0.118 nats in s0 vs +27–30 nats in s1/s3). This reflects the curriculum training protocol making routing more concentrated — a few hub blocks dominate many categories simultaneously. Ablating 5 top-lift blocks in s1–s3 essentially removes blocks that all categories rely on heavily, producing large absolute effects. The category-specific signal (self − other differential) is correspondingly harder to isolate but still present for causality and dialogue.

Data source: `scripts/competence_ablation_crossseed.py` → `data/eval/phase1/competence_ablation_s{1,2,3}.json`

3.7 Summary

SR-Core has WS=k by construction (routing fixed at r=1; Chapter 2). What these free-routing scaling experiments add is that the surrounding conditions stay healthy as the bank grows:

- Quality does not degrade — and modestly improves — with increasing n at fixed k
- Routing remains healthy (no collapse) at all tested bank sizes
- Routing specialization is stable across seeds

(Note: these are pre-SR-Core free-routing variants with unique-blocks/token ≈6–7, not WS=k=4; they validate that bank scaling is benign, not the WS bound itself — that is architectural.)

Sparse routing C outperforms dense A at equal compute, and pure recursion B is strictly dominated — validating the architectural motivation for SR-Core.

Data sources: `data/eval/phase1/tinystories_b*.json`, `C_routed_s*.json`, `competence_b64k4R6_*.json`

Chapter 4: Dispatch Overhead — The CPU-Side Tax of Sparse Routing

4.1 Motivation

The working-set guarantee (WS=k) reduces the number of blocks that must be loaded per token. However, loading fewer blocks is only beneficial if the cost of *selecting* which blocks to load (dispatch overhead) does not dominate the savings.

This chapter characterizes the dispatch overhead of SR-Core relative to a compute-matched dense baseline on CPU, to establish a realistic picture of where the current prototype stands and what hardware properties are required for the sparse approach to yield net inference speedup.

4.2 Benchmark Setup

Hardware: Intel Core i7-10700 (8 cores / 16 threads, 2.90 GHz), 16 GB RAM; PyTorch CPU backend
Config: batch size 8, sequence length 128 (1,024 tokens/forward), median over 15 runs
Note: Core time excludes deep-supervision readout heads; same overhead for all models.

Four measurement modes for the sparse model:

- **A_dense_all:** All n=64 blocks × R=6 steps = 384 block-applications (upper bound)
- **B_sparse_route:** True sparse dispatch, full routing at each step r
- **C_sparse_pre:** True sparse dispatch, routing pre-computed once at r=1 (= SR-Core)
- **D_fixed_core:** True sparse dispatch, completely fixed block set (no routing)

4.3 Results

Model	Block-apps/token	L_final	Params	Core time	tok/s
Sparse b64 R=6 (C_sparse_pre)	24	3.720	19M	**158 ms**	6,471
Sparse b64 R=2	8	~3.72	19M	57 ms	18,057
Dense d24 (compute-matched)	**24**	**3.481**	**8.7M**	**56 ms**	**18,185**
Dense d8	8	3.582	4.5M	18 ms	58,016

Additional breakdown at R=6:

- A_dense_all (64 blocks × 6 = 384 apps): 823 ms

- D_fixed_core (~7 fixed blocks × 6 = ~42 apps): 92 ms
- Router overhead (B − C): **7 ms** — only 5% of sparse total; routing is not the bottleneck
- Pure dispatch cost: 151 ms for 24 block-apps vs. 56 ms dense → **2.7× dispatch tax**

Result: Sparse R=6 is **2.8× slower** than compute-matched Dense d24 at *identical* 24 block-applications per token (158 ms vs. 56 ms), and qualitatively worse (3.720 vs. 3.481) despite more parameters (19M vs. 8.7M).

This is a measured result, not a theoretical estimate. The overhead is confirmed.

4.4 Sources of Dispatch Overhead

Three main contributors:

1. **Routing computation:** The router must evaluate all n block keys against the query vector and apply top-k selection. This is O(n) per token.
2. **Gather/scatter:** After routing, the selected k blocks must be gathered from the block bank (non-contiguous memory access). Dense execution is a single contiguous matrix multiply.
3. **Kernel launch overhead:** Each block is a separate operation. At k=8 and R=6, this is 48 small matrix multiplications per token vs. a single large one for dense.

4.5 Implications

The 2.8× dispatch overhead establishes a clear engineering requirement: the bytes-in-motion reduction must exceed 2.8× *on the transfer-bound path* before sparse routing yields net benefit.

From the offloading simulation (Chapter 5b):

- At K=8 cached blocks, SR-Core requires 4.1× fewer bytes/token than dense layer-offloading (3,035 vs 12,348 KB/token)

In the RAM→VRAM transfer-bound regime, the 4.1× transfer reduction can in principle offset the 2.8× dispatch overhead — but this depends on the ratio of transfer latency to compute latency, which varies by hardware. On memory-bandwidth-limited hardware (where transfers are the bottleneck), the sparse approach can win; on compute-bandwidth-limited hardware, it cannot.

The dispatch overhead does not invalidate the sparse approach; it specifies the target deployment regime: hardware where the RAM→VRAM transfer cost dominates, and the overhead of dynamic dispatch is amortized by large block sizes and high bandwidth.

4.6 Mitigation Directions

1. **Larger blocks:** Higher arithmetic intensity (FLOPs/byte) amortizes dispatch cost over more computation. Attention blocks have ~128× higher intensity than MLP blocks at typical sequence lengths.
2. **Hardware support:** GPU-native sparse dispatch (e.g., CUDA Streams, structured sparsity) eliminates the Python-level dispatch overhead.
3. **Batch processing:** At inference batch size >1, routing decisions can be vectorized. The overhead is amortized across the batch.
4. **Leiterbahn (future):** If the active block set is known from a routing index before execution begins, dispatch reduces to a table lookup with prefetched weights.

4.7 Summary

Sparse dispatch imposes a ~2.8× CPU overhead relative to compute-matched dense execution. This overhead is *not* disqualifying in the target deployment scenario (RAM→VRAM transfer-bound inference), where the 4.1× reduction in bytes-in-motion exceeds it. However, a real-latency demonstration requires hardware where transfer costs dominate, which the current CPU prototype does not test.

Data source: `data/eval/phase1/cpu_benchmark.json`

Chapter 5: HeteroMini Experiments

This chapter covers three experiments on the HeteroMini-v1 corpus: a multi-domain quality matrix (5a), an offloading simulation (5b), and cross-seed robustness of the routing structure (5c).

5a: Multi-Domain Quality Matrix

5a.1 HeteroMini-v1 Corpus

HeteroMini-v1 is a four-domain pretraining corpus constructed for this work:

Domain	Source	Tokens (~)
Web	FineWeb-Edu	~1.6M
Wikipedia	Wikipedia dumps	~1.6M
Code	The Stack (Python)	~1.6M
Literature	Project Gutenberg	~1.6M
Total		**~6.6M**

Tokenized with a shared byte-level BPE vocabulary of 8,000 tokens. Documents are kept contiguous; the evaluation split has no training overlap.

The four-domain design tests whether SR-Core routing generalizes across heterogeneous input distributions, and whether held-out losses differ meaningfully by domain (as expected if routing specialization mirrors domain structure).

5a.2 Model Variants and Matrix Results

The matrix is a 2,000-step smoke run comparing 8 model configurations to establish which architectural variants warrant longer training. All except SR-Core b64 R6 (10,000 steps) are short-run comparisons.

Model	Steps	Params	L_final	Anytime	WS	Cache@8	Domain acc
Dense d24	2,000	8.7M	6.473	0.033	—	—	—
Dense d8	2,000	4.5M	6.541	0.006	—	—	—
Naked b32 R2	2,000	10.8M	6.756	0.000	4.13	15.6%	45.4%
Naked b32 R6	2,000	10.8M	6.568	0.022	7.49	15.9%	49.4%
SR-Core b32 R2	2,000	10.8M	6.561	0.000	**4.0**	13.3%	47.3%

Model	Steps	Params	L_final	Anytime	WS	Cache@8	Domain acc
SR-Core b32 R6	2,000	10.8M	6.710	0.022	**4.0**	2.0%	45.0%
SR-Core b64 R2	2,000	19.3M	6.659	0.000	**4.0**	17.1%	46.9%
SR-Core b64 R6	**10,000**	**19.3M**	**5.371**	**0.017**	**8.0**	**12.4%**	**61.6%**

(WS = working_set.contiguous; Cache@8 = cache miss rate at K=8; Domain acc = routing classifier accuracy; chance = 25%)

Key observations from the matrix:

This matrix is a smoke run, not a quality ranking. At 2,000 steps no variant has converged; the table only identifies which configurations warrant longer training. Some orderings change with sustained training (Section 5a.5), so do not read row-vs-row L_final differences here as architectural verdicts — but the overall direction holds: Dense remains the quality ceiling once trained (5a.5).

SR-Core b32 R6 at 2,000 steps appears worse than naked b32 R6 (6.710 vs. 6.568). This is a training warm-up artifact, not a fundamental quality issue: routing with load-balancing takes more steps to settle. The b32 R6 trajectory bears this out — dead blocks fall 6 → 8 → 3 → 0 → 0 from step 1k to 10k, i.e. the early "collapse" resolves itself with training rather than being permanent.

WS=k holds exactly for all SR-Core variants (working_set.contiguous = k in all configurations). Naked variants show WS > k because they use free routing.

SR-Core b32 R6 has the lowest cache miss rate (2.0% at K=8) — the routing collapses early (it is in the low-diversity regime at this step count), leading to high cache hit rates but likely degraded quality from under-specialization.

Domain routing classification accuracy: b64 R6 at 10k steps achieves 61.6% vs. 25% chance — routing does encode domain identity. All 2k-step variants are 45–49% (barely above chance), confirming that domain specialization requires sustained training.

5a.3 Long-Run b64 R6 (10,000 Steps)

The b64 R6 model trained for 10,000 steps:

- L_final = 5.371 (significantly better than the 2k-step runs at ~6.5–6.7)
- Domain Jaccard off-diagonal = 0.225 (cross-domain routing overlap)
- Domain Jaccard matrix: Web–Lit = 0.60 (high similarity), Code–others ≈ 0.07–0.14 (Code routing is highly distinct from all other domains)
- Cache miss rate K=8: 12.4% contiguous, 13.9% shuffled — locality benefit from contiguous document batching

The Code domain has the most distinct routing signature (Jaccard with others ≈ 0.07), consistent with its linguistic distinctiveness in the corpus.

5a.4 Seen vs. Unknown Generalization

Generalization gap for Dense d24@10000 on held-out documents not in the training split:

Split	Loss	PPL	Top-1 acc	Anytime gain
Seen	5.133	169.6	18.5%	0.028
Unknown	5.287	197.7	17.3%	0.013
Gap	**+0.154**	**×1.17**	−1.2 pp	−0.015

Per-domain breakdown (seen / unknown loss):

Domain	Seen	Unknown	Gap
Web	5.481	5.713	+0.232
Wikipedia	5.421	5.623	+0.202
Code	**4.407**	**4.700**	+0.293
Literature	5.165	5.205	+0.040

The generalization gap is smallest for Literature (+0.040) and largest for Code (+0.293), suggesting Code is both the easiest domain in absolute terms and the most sensitive to distribution shift. The overall PPL ratio (1.17×) is consistent with a model trained on 6.6M tokens of narrow-domain data.

5a.5 Quality: Dense Is the Ceiling

The 2,000-step matrix (5a.2) is a smoke run, not a final ranking. With sustained training, the final-step loss of SR-Core b64 k8 R6 is reproducible across four independent seeds. All values below are the mean cross-entropy at the final recursion step r=R, re-measured under one fixed protocol (60×16×128 tokens, contiguous windows) on both the training-domain (seen) corpus and the held-out split:

Seed	L (seen)	L (held-out)
s0	5.334	5.491
s1 (low-loss outlier)	4.989	5.291
s2	5.283	5.464
s3	5.290	5.518
Mean ± std	**5.224 ± 0.137**	**5.441 ± 0.089**
Mean excl. s1	5.302	5.491

Seed s1 is a low-loss outlier (also noted in Chapter 6); excluding it, the three remaining seeds cluster tightly (seen 5.28–5.33). Dense d24 at 17k steps, under the *same* protocol, reaches **seen 4.793 / held-out 5.102**. SR-Core therefore trails dense by **~0.5 nats on the seen contiguous loss (0.51 excluding s1) and ~0.34 nats held-out** — with half the block-applications per token (24 vs. k·R = 48). At this scale **Dense d24 is the quality ceiling; SR-Core does not match it.**

Data source: `data/eval/heteromini/eval_quality_four_seed_summary.json` — regenerate with `python monograph/scripts/eval_four_seed_quality.py --entropy`. (The earlier version of this table reported a single ”held-out” column that was in fact the seen-corpus loss and used a per-run eval; the numbers above supersede it under one auditable protocol.)

The observed gap is not explained by parameter count or block-application compute. A *param- and compute-matched* comparison settles this directly. A SR-Core variant sized to Dense d24 — n=64, k=16, R=4, block_hidden=192 — has 8.75M parameters (vs. d24's 8.70M) and performs exactly **6.29M parameter-applications per token** (k·R·block = 64·98k), identical to d24's 24·262k. Same parameters, same compute, same backbone. Under the same protocol as above, across three seeds it reaches **seen 5.269 ± 0.020 / held-out 5.459 ± 0.007 — still ~0.48 nats (seen) / ~0.36 nats (held-out) behind Dense d24**. Matching both parameters *and* compute does **not** close the gap. The evidence points to the SR-Core *format* (narrow active set + weight-tied recursion) as the source — the gap persists even when both budgets are matched. Note this matched variant is also *tighter* across seeds

than the plain k8 R6 set (std 0.02 vs 0.14, no low-loss outlier), so it is the cleaner evidence. Whether the gap narrows at convergence or at larger scale remains open (Section 7.5.6).

Data source: same summary file (`param_matched_k16_R4` block in `data/eval/heteromini/eval_quality_four_seed_summ`

The mechanism is a breadth-vs-depth tradeoff at fixed compute: dense brings ~6.3M *distinct* parameters to bear per token (each applied once), while this SR-Core touches only ~1.6M distinct parameters (k=16 blocks) and reuses them R=4 times via recursion. Recursion recovers only part of the capacity that breadth would provide — Section 5d measures exactly how far it recovers, and the answer (a saturating gain by r≈4) is consistent with this residual ~0.5-nat gap.

This number is a lower bound, not a converged value. At 15k steps both models are still descending (~0.2 nats per 2,500 steps each); 6.6M tokens is a small corpus and neither has converged. The descent *rates* are comparable, so this data shows no evidence that SR-Core closes the gap with more training — but it cannot be ruled out without a longer run (a convergence study is the natural next experiment).

The conclusion matches the smoke run's direction and the rest of this work: **SR-Core's contribution is transfer efficiency, not language quality.** It does not win on held-out loss — it transfers k=8 instead of 24 blocks per token (5b) and converts that into measured throughput (5b.4). The argument rests on the transfer reduction, never on a quality claim.

5b: Offloading Simulation

5b.0 Why the Premise Is Bandwidth, Not Compute

Every comparison in this chapter must be read under the deployment scenario the architecture targets: a model too large for VRAM, held in host RAM, streaming k blocks per token across PCIe to the GPU. In this regime the GPU is **memory-bandwidth-bound** — it stalls waiting for block transfers, not for arithmetic. Compute performed on blocks already resident in VRAM can be **partially hidden** when transfer dominates and the sparse dispatch is fused (Chapter 5b.4 shows the naive, unfused version is *not* hidden — it is kernel-launch-bound and loses to dense until the per-block launches are grouped).

This reframes the compute asymmetry from Section 5a.5. SR-Core's 48 block-applications per token (vs. dense d24's 24) are not a cost in this scenario: they execute in wall-clock time the dense model would spend idle on its larger transfer. The quantity that decides throughput here is **bytes moved per token**, not FLOPs per token — and that is exactly what the offloading simulation measures. A model that moves fewer bytes per token serves more tokens per second whenever PCIe, not the ALU, is the bottleneck.

The corollary defines what SR-Core must deliver to be useful in this regime: not a quality win, but a quality that is *close enough* to justify the transfer saving. At this scale SR-Core trails Dense d24 by ~0.5 nats (5a.5) while moving k=8 instead of 24 blocks per token (5b.3) — whether that trade is worthwhile depends on how scarce bandwidth is in the target deployment, and on whether the gap narrows at scale or with matched compute (both untested).

5b.1 Simulation Setup

The LRU offloading simulator estimates bytes-in-motion under a simple caching model:

- A virtual cache of capacity K blocks (K ∈ {8, 16, 24, 32}) is maintained per inference sequence
- On each token, the k active blocks are checked against the cache
- Cache misses incur a transfer cost of block_size bytes

- LRU eviction policy

This models the RAM→VRAM transfer scenario where frequently-used blocks remain resident.

Simulator runs on 17,000-step checkpoints (after full training, before entropy continuation).

5b.2 Baseline Comparison

Dense layer-offloading baseline: at cache capacity K, a dense model with n_layers=24 must load all layers that are not cached. At K < 24, every token triggers at least n_layers - K cache misses.

For dense d24 at any K < 24:

```
bytes/token ≈ (n_layers - K) × layer_size
```

At K=8: dense requires 16 layer transfers per token (no blocks resident). At K≥24: dense fits entirely in cache → 0 transfers. This crossover is an important boundary condition: for K≥n_layers, dense wins.

5b.3 Results

Full cache-sweep (contiguous document order, FP16):

Model	K=4 KB/tok	K=8 KB/tok	K=16 KB/tok	K=32 KB/tok
Dense d24	12,348	12,348	12,348	~1
SR-Core ctrl	**24,694**	3,034	1,857	568
+ entmin λ=0.003	24,694	3,015	1,808	520
+ entmin λ=0.005	24,694	2,973	1,747	**489**

(Source: `offload_sim_17k.json`, `bytes_per_token_fp16 / 1024`)

K=4 reversal: At K=4, sparse is ~2× *worse* than dense (24,694 vs 12,348 KB/tok). The WS=k=8 working set does not fit in a K=4 cache — effectively all 8 blocks miss on every token. This is an important boundary condition: the sparse advantage only holds for K ≥ k.

K=8 breakeven: The cache exactly fits the WS=k=8 working set. SR-Core ctrl uses 4.1× fewer bytes/token than dense (3,034 vs 12,348 KB/tok). Entropy consolidation (λ=0.005) adds a further 2% reduction (3,034 → 2,973 KB/tok).

K=16–32: Sparse bytes/token continues to fall (hub blocks become resident across tokens). At K=32: ctrl uses 568 KB/tok vs dense's 12,348 KB/tok (K=8 reference) — but dense at K=32 also fully fits in cache (~1 KB/tok). The sparse advantage vs. dense **disappears** above K≈24 because dense d24 fits entirely in cache once K ≥ n_layers.

λ=0.005 is the best model at every K level, confirming that entropy consolidation monotonically reduces bytes-in-motion across the entire cache regime.

These are *simulated* estimates assuming LRU eviction and contiguous document order. Real transfer costs depend on DRAM bandwidth, PCIe characteristics, block granularity, and prefetch scheduling — none of which are modeled here.

5b.4 First Real-Hardware Measurement (RTX 2060)

A first prototype replaces the simulator's assumed bandwidth with measured RAM→VRAM transfer (block weights pinned in host RAM, streamed to a K-slot VRAM cache; design and code: `docs/streaming_prototype.md`, `experiments/streaming_prototype.py`). Two findings:

1. **The simulated bytes/token reproduce exactly** on real hardware (the streaming engine's fetch counts match the LRU misses of `offload_sim` at every K), and the **measured H2D bandwidth is 11.4 GB/s** — i.e. the simulator's 16 GB/s assumption was ~40% optimistic. On the transfer axis the advantage holds: SR-Core moves 6.7× fewer bytes/token than dense layer-offloading and therefore spends 6.7× less transfer time (166 vs. 1112 µs/token).
2. **The wall-clock throughput advantage appears once per-block launch overhead is removed.** A first (naive, per-block) implementation was kernel-launch-bound — ≈12 ms/token dominated by the 48 small block-applications (k=8 × R=6), with transfer <2% of the total — and was *slower* than dense (81 vs. 105 tok/s). Replacing the k per-block calls of each recursion step with a single **grouped block matmul** gives a 3.8× speedup (78 → 297 tok/s) and reverses the ranking: **SR-Core 297 tok/s vs. dense layer-offloading 205 tok/s — a 1.45× measured wall-clock advantage.** At a VRAM budget K=16 < D=24 the dense baseline thrashes (reloads all 24 layers/token, 12.3 MB) and becomes transfer-bound, while SR-Core moves 6× less (4 fetches/token, 2.1 MB). The remaining gap to the all-resident compute ceiling (297 → 364 tok/s) is the now-visible transfer cost (~18% of the time), which a two-stream overlap (prefetch the next token's blocks during the current token's compute) recovers in part — a further 5–9% in the transfer-heavy regime, reaching ~1.6× over dense on the same hardware, and correctly nothing once the cache is large enough that transfer is already free.

This closes — at small scale and as a real measurement — the chain *fewer bytes → more tokens/second* that Chapter 5b only simulated. It also sharpens the central premise: SR-Core trades *more compute* (2× the block-applications) for *less transfer*; that trade is invisible when the compute path is launch-bound and only pays off once the path is efficient and the dense baseline is transfer-bound (model larger than the VRAM cache). The 1.45× is modest at this toy scale (514 KB blocks, forced streaming, batch-1, no overlap); the scale projection (Section 5b, `--project`) into the regime of large blocks and scarce bandwidth extrapolates a substantially larger advantage but remains a *projection beyond the measured point*.

5c: Cross-Seed Robustness

5c.1 Setup

Three independently initialized SR-Core b32 k8 R6 models (seeds 0, 1, and 2) are trained on the same HeteroMini-v1 corpus for 15,000 steps.

Question: is the routing structure (Jaccard overlap between seeds, domain specialization pattern) reproducible across random initializations?

5c.2 Results

Model: `srcore_b32_k8_R6@15000`, 3 independently initialized seeds, 81,920 eval tokens each.

Seed	Unique cores	Top-1 cov.	Gini	Dead	Cache miss K=16	Domain Jaccard
s0	7,738	2.6%	0.288	0	5.3%	0.292
s1	14,445	1.2%	0.256	0	5.6%	0.216
s2	4,889	3.0%	0.206	0	6.7%	0.376

(Unique cores = number of distinct active-block tuples; Gini = load imbalance 0=flat 1=one block; Domain Jaccard = mean off-diagonal routing overlap between domains)

What is robust across seeds:

- Zero dead blocks (WS=k holds, routing health is stable)
- Cache miss K=16 is 5–7% across all seeds (LRU cache works at this K)
- Gini coefficient is low (0.21–0.29) — no single block dominates

What varies across seeds:

- Unique core count ranges 3× (4,889 to 14,445) — routing diversity is seed-dependent
- Domain Jaccard spans 0.216–0.376 — the degree of domain specialization is not stable
- s2 has the fewest unique cores (more repetitive active sets) and highest domain overlap

The seed-to-seed variation in routing diversity is the key finding: the WS=k guarantee and routing health are seed-invariant, but the *structure* of routing specialization (which domains activate which blocks) is initialization-dependent. Blocks learn domain functions, but which block learns which function is arbitrary.

5c.3 Anytime Inference

Anytime gain = L(r=1) − L(r=R): quality improvement from early exit (r=1) to full recursion (r=R=6). Evaluated on **seen** documents; `srcore_b32_k8_R6@15000`.

r	Block-apps	s0 L_seen	s1 L_seen	s2 L_seen	Code gain s0	Code gain s1	Code gain s
1	8	5.263	5.112	5.051	0.000	0.000	0.000
2	16	5.312	5.081	5.045	0.026	0.024	0.089
3	24	5.298	5.072	4.979	0.049	0.038	0.149
4	32	5.198	5.058	4.902	0.059	0.048	0.187
6	48	5.235	5.045	4.886	0.053	0.055	0.204
Gain r1→r6		**+0.028**	**+0.067**	**+0.165**			

(Positive anytime gain means deeper recursion improves quality; s0 total gain is actually slightly negative at r=2/3 before recovering — non-monotone depth curve.)

Key observations:

- **Seed 2 has 6× larger anytime gain than seed 0** (0.165 vs. 0.028). Anytime depth benefit is not stable across seeds at this step count.
- **Code benefits most from depth**: code_gain increases monotonically with r in all seeds. At r=6, Code gain is 0.053–0.204 nats (0.053 for s0, 0.204 for s2).
- **s0 has a non-monotone depth curve** (L increases at r=2 before decreasing at r=4). This is a training instability artifact — the deep-supervision weighting has not fully resolved by step 15,000 for this seed.
- Throughput is seed-independent (~4,300–4,900 tok/s GPU) since block-apps are fixed.

The anytime property is present but seed-dependent at 15,000 steps. Longer training (as demonstrated by b64 R6 at 10,000 steps in section 5a.3) stabilizes the gain.

5c.4 Domain Partition Analysis

Domain-specific routing specialization is measured by the Domain Jaccard off-diagonal: how much routing overlap exists between different domains. Lower = more specialized.

From Section 5c.2:

- s0: Domain Jaccard = 0.292
- s1: Domain Jaccard = 0.216 (most specialized)

- s2: Domain Jaccard = 0.376 (least specialized — most shared routing between domains)

This seed-dependence is expected: which block specializes for Code vs. Literature is arbitrary; what matters is that *some* domain partition exists (all values < 0.5), not that the same blocks are responsible across seeds.

Quantitative domain partition analysis (which specific blocks concentrate on which domain) is initialization-sensitive and not reported as a cross-seed aggregate.

5d: Recursion Depth vs. Active Capacity (A-Sweep)

Section 5a.5 attributes the dense–sparse quality gap to the SR-Core format trading *breadth* (distinct active parameters per token) for a small streamable working set, with recursion only partly compensating. This section measures how far recursion compensates, as a function of the active-brain size **A = k · block_size** — the distinct parameters a token activates per step.

Block size is held fixed (n=64, R=6, block ≈ 263k params) and k is swept ∈ {2, 4, 8, 16} on HeteroMini (15k steps), so A ranges 0.5M–4.2M. The per-recursion-step gain L(r=1) − L(r) is read post-hoc from the deep-supervision readout (no retraining; ”R is inference-time”):

k	A (active params)	useful depth	total anytime gain (seen)	best L
2	0.53M	r≈1 (dead)	0.0015	5.33
4	1.05M	r≈4	0.0595	5.46
8	2.10M	r≈2–4 (seed-dep.)	0.008 / 0.017 / 0.031 (3 seeds)	5.21–5.34
16	4.20M	r≈4	0.0625	5.24

Two clean signals survive, and one honest caveat:

A recursion floor exists. At k=2 (A=0.53M) recursion is dead — gains vanish by r≈2. The block sits at its representational ceiling at r=1 and cannot encode a refinement of its own output. Above the floor (A $\gtrsim$ 1M, k ≥ 4) recursion lives.

Useful depth saturates at r≈4. Wherever recursion is alive, the per-step gain is exhausted by r≈4; steps r=5–6 add essentially nothing. This is the empirical basis for the R=4 choice in the param-matched experiment (5a.5) — and it bounds how much recursion can substitute for breadth.

Magnitude is seed-dominated — do not over-read it. Total recursion gain is *not* monotone in A (k=4 and k=16 both ~0.06, k=8 only ~0.017), and a three-seed replication of k=8 gave a 4× spread (0.008 / 0.017 / 0.031). With one seed per point the sweep is underpowered to rank neighbouring A values by gain magnitude; only the floor (k=2 dead) and the depth saturation (r≈4) survive the seed noise. Reading a precise ”optimal A” off the magnitude curve would repeat the single-seed overinterpretation this work elsewhere flags.

The design conclusion is coarse but robust: keep A above the floor (~1M, k ≥ 4) and use R≈4; the precise sweet-spot is below the resolution of a single-seed sweep. This caps how far recursion recovers the capacity that breadth would provide — consistent with the residual ~0.5-nat gap to dense at matched compute (5a.5).

Data source: `data/eval/heteromini/anytime_inference_srcore_b64_*@15000_s*.json`; figure: `figures/fig_asweep_depth.png` (script: `scripts/plot_asweep.py`).

5e: Summary

HeteroMini experiments establish:

1. **Dense d24 is the quality ceiling; the ~0.5-nat gap is not explained by parameter or compute budget.** Under one fixed protocol (Section 5a.5), SR-Core b64 k8 R6 reaches seen 5.30 / held-out 5.44 (three seeds excl. a low-loss outlier) vs. Dense d24's seen 4.79 / held-out 5.10. A param- *and* compute-matched SR-Core (8.75M params, 6.29M apps/token, identical to d24) still trails by **~0.48 nats seen / ~0.36 held-out** (seen 5.269 ± 0.020; Section 5a.5) — so the gap is not a budget artifact; the evidence points to the narrow-active-set + recursion format. Both models are still descending at their stops, so this is a lower bound, not a converged value; whether the gap narrows at convergence or scale remains open (Section 7.5.6). SR-Core's contribution is transfer efficiency, not quality.
2. **Recursion has a measurable useful-depth band** (Section 5d): below an active-brain floor (A ≈ 1M) recursion is dead; above it, per-step gains saturate by r≈4. Magnitude is seed-dominated and not a reliable design signal.
3. **SR-Core scales in bank size:** b64 R6 outperforms b32 R6 across domains, confirming that the fixed active budget k=8 benefits from a larger block vocabulary.
4. **Offloading simulation shows 4.1× transfer reduction** vs. dense layer-offloading at K=8, and a first real RAM→VRAM prototype turns it into a measured wall-clock win (~1.6× over dense layer-offloading on an RTX 2060; Section 5b.4). This — not quality — is the load-bearing result: in the bandwidth-bound regime, paying ~0.5 nats to move k=8 instead of 24 blocks per token is the trade.
5. **Cross-seed training is robust:** key routing properties (entropy, Jaccard structure) replicate across independently initialized models.

Data sources: `data/eval/heteromini/`, `data/eval/phase1/offload_sim.json`, `data/eval/entmin/offload_sim_17k.json`

Chapter 6: Entropy-based Router Consolidation

This chapter is the primary experimental contribution of this monograph. It introduces entropy-based router consolidation as a controllable axis for cache efficiency and characterizes its effects on routing structure, language quality, and simulated offloading cost.

This chapter is self-contained: every numerical claim is backed by an eval JSON file in `data/eval/entmin/`, and the figures are regenerated from those files by `scripts/build_figures.py`.

6.1 Motivation: Routing Collapse vs. Routing Consolidation

SR-Core guarantees WS=k (Chapter 2), but does not control *which* k blocks a token activates. Across training, the routing distribution can behave in two extreme ways:

- **Diffuse routing:** Each token selects a near-random subset of k blocks from the full bank. High cache miss rate — new blocks must be loaded for almost every token.
- **Consolidated routing:** Many tokens select the same or overlapping k-subsets. A small number of blocks are heavily shared. Low cache miss rate — the active set is largely resident after a short warm-up.

From the offloading perspective, consolidated routing is desirable: a shared LRU cache benefits from routing patterns where the same blocks are repeatedly active.

However, extreme consolidation — all tokens selecting the same k blocks — would collapse the block bank to a small subset and eliminate the architectural benefit of having n » k. The target is *controlled consolidation*: more concentrated than random, less concentrated than collapse.

6.2 The Entropy Objective

6.2.1 Loss Function

The full training loss is:

```
L = L_LM + λ · H(p_t^(1))
```

where L_LM is the standard next-token cross-entropy, and H(p_t^(1)) is the router entropy at step r=1 for token t:

```
H(p) = -Σ_{b=1}^{n} p_b log p_b
```

The entropy penalty pushes the pre-selection distribution p toward concentration — lower entropy means the router is more certain about which blocks to select.

6.2.2 Why r=1?

Routing is fixed at r=1 and reused for r=2…R (SR-Core constraint). Therefore, applying the entropy objective at r=1 is the only point where routing can be influenced. Applying it at deeper steps would have no effect on the selection.

6.2.3 Why pre-topk?

The entropy H(p) is computed on the full pre-topk distribution p ∈ ℝ^n, before the discrete top-k selection. This preserves the differentiable path through the router: the top-k operation is not differentiable, but H(p) is. Applying the penalty to the pre-topk distribution allows gradients to flow through the router without straight-through estimators or other approximations.

6.2.4 λ Hyperparameter

λ controls the strength of entropy pressure. The λ sweep covers: λ ∈ {0.001, 0.003, 0.005, 0.007}

plus the unregularized continuation (ctrl, λ=0.000) and two target-entropy variants (λ·relu(H(p) - H_target) with H_target ∈ {3.75, 3.70}).

6.3 Experimental Setup

6.3.1 Training Protocol

All SR-Core models (b64, k=8, R=6) are trained for 15,000 steps as a shared base on HeteroMini-v1. The entropy continuation is then applied for 2,000 additional steps from this shared checkpoint, at each λ value independently.

The unregularized continuation (ctrl) runs for the same 2,000 steps without the entropy objective. This is the direct comparison baseline.

Cross-seed validation: λ ∈ {0.003, 0.005} are evaluated at two independent seeds (s0 and s1, independently initialized base models).

6.3.2 Negative Controls

Three negative controls validate that the entropy objective is the correct intervention:

- **Softfull:** Soft Jaccard penalty on router probability vectors across *consecutive tokens* — encourages cross-token routing similarity rather than within-token concentration. If this also consolidates routing, it suggests the effect is not specific to entropy.
- **Softsharp_a2:** Temperature sharpening (α=2) applied to router logits.
- **Reduced_noise:** Lower Gumbel noise (σ=0.1) during top-k sampling.

6.3.3 Evaluation Protocol

Router consolidation metrics are measured on 40 held-out batches via eval_compare.py:

- Router entropy H(p): lower = more concentrated pre-selection

- Unique core combinations: distinct active k-subsets per evaluation run
- Hard-overlap Jaccard: mean Jaccard across consecutive tokens (higher = more stable paths)
- LRU bytes/token at K ∈ {8, 16, 24, 32}: simulated cache cost

Language quality metrics use paired evaluation (same held-out batches, dense d24 as consistent anchor per run):

- Per-domain held-out loss: Web, Wikipedia, Code, Literature
- Code-ratio: code_loss / mean(web_loss, wiki_loss, lit_loss) — lower = better code generalization relative to other domains

6.4 Results: Router Consolidation

6.4.1 Monotonic Response

All four routing metrics respond monotonically to increasing λ:

λ	H(p)	Unique cores	Hard overlap	K=24 KB/tok
0.000 (ctrl)	3.846	25,853	0.251	1,103
0.001	3.815	25,399	0.254	1,098
0.003	3.799	23,077	0.262	1,085
0.005	3.732	21,273	0.266	1,020
0.007	3.647	18,817	0.278	1,009

The response is non-linear: most consolidation occurs between λ=0.001 and λ=0.005; gains from 0.005 to 0.007 are smaller. Three qualitative regimes are identified:

- **Generalization-balanced (λ ≤ 0.003):** Moderate consolidation, quality effects either neutral or positive.
- **Cache sweet spot (λ ≈ 0.005):** Largest consistent cache improvement, quality effects seed-dependent.
- **Boundary (λ = 0.007):** Maximum consolidation, quality picture less predictable.

6.4.2 Negative Control: Softfull

The softfull objective moves routing metrics in the *opposite* direction:

Metric	ctrl	softfull	Direction
Router entropy	3.843	3.868	+0.025 (worse)
Unique cores	25,684	30,246	**+17.8%** (worse)
Hard overlap Jaccard	0.259	0.251	−0.008 (worse)
K=24 LRU KB/tok	—	—	<1.4% change

Cross-token similarity objectives act on pairwise overlap but do not control the within-token distribution concentration. A Jaccard penalty can be satisfied by increasing routing diversity (more distinct combinations with higher pairwise overlap on average), which moves metrics in the wrong direction for cache efficiency.

This confirms that **pre-selection entropy is the correct intervention point** — not cross-token similarity, not temperature sharpening.

6.5 Results: Cache–Quality Pareto

6.5.1 λ=0.003 Pareto-Dominates ctrl

In seed 0:

- K=24 cache: 1,103 → 1,085 KB/tok (−1.6%)
- Code-ratio: 0.886 → 0.866 (−0.021, improvement)

In seed 1:

- K=24 cache: not separately measured (offload_sim aggregates by architecture, not λ-variant; see 4-seed update below)
- Code-ratio: 0.896 → 0.822 (−0.074, improvement)

λ=0.003 is a **Pareto improvement**: both cache efficiency and code generalization improve relative to the unregularized baseline, replicated across two independent seeds.

4-seed update. Two further seeds (s2, s3) for both λ=0.003 and λ=0.005 are now trained, extending the sweep to four seeds. Re-measured under the fixed protocol of Section 5a.5, the seen final-step loss is consistent across the sweep (λ=0.003: s0=5.328, s2=5.147, s3=5.138, with the known low-loss outlier s1=4.986; λ=0.005 within 0.003 of these — the two λ values are nearly indistinguishable on quality). The *quality* side of the Pareto thus has 4-seed support. The per-seed **cache** table (the Pareto x-axis) was not recomputed for s2/s3: `offload_sim` labels checkpoints by architecture, not by λ-variant, so a single pass cannot separate them — the cache side remains at two seeds. This is the one acknowledged loose end in the entropy chapter.

Data source: `entmin_lam003` / `entmin_lam005` blocks in `data/eval/heteromini/eval_quality_four_seed_summary.json`.

6.5.2 λ=0.005: Larger Cache Gain, Seed-Dependent Quality

Largest consistent cache improvement:

- K=24: 1,103 → 1,020 KB/tok (−7.5%)
- K=16: −5.2%

Language quality is seed-dependent (within-run comparison):

- Seed 0: code-ratio 0.877 vs. within-run ctrl 0.859 (Δ = +0.018, slight degradation)
- Seed 1: code-ratio 0.841 vs. within-run ctrl 0.863 (Δ = −0.022, improvement)

The within-run ctrl values differ across seeds due to batch-sampling variance (~±0.05 nats). A single ctrl anchor (0.886, from the λ=0.003 run) is used for the Pareto figure for visual consistency; within-run comparisons give the true paired delta.

λ=0.005 is cache-best among the tested configurations, but the quality effect is not reliable across seeds and should not be characterized as either a cost or a benefit.

6.5.3 λ=0.007: Boundary

Further cache improvement: K=24 1,009 KB/tok (~1.1% over λ=0.005). Code-ratio returns toward ctrl levels (0.884 in seed 0).

λ=0.007 is treated as the boundary point of the controllable axis.

6.5.4 Target-Entropy Variants

Bounded objective: λ·relu(H(p) − H_target), targets H=3.75 and H=3.70. Both fall below the Pareto frontier of the unconstrained sweep: slower convergence in the 2,000-step window, does not reach the same cache/quality operating points.

6.6 Results: Language Quality vs. Dense

Model	Web	Wiki	Code	Lit
Dense d24	5.448	5.456	4.621	4.735
SR-Core ctrl	5.807	5.841	4.856	5.286
+ entmin λ=0.003	5.672	5.688	4.969	5.352
+ entmin λ=0.005	5.887	5.792	4.985	5.419

Dense d24 is the quality upper bound in all domains. The dense advantage is largest in Literature (0.55 nats) and smallest in Code (0.24 nats).

Entropy minimization does not narrow the dense–sparse quality gap. The objective shapes routing structure; it does not trade quality for cache efficiency in either direction at λ=0.003 (no tradeoff measured). At λ=0.005 the effect on absolute loss is small and mixed.

6.7 Results: Offloading Simulation

See Chapter 5b for the full offloading simulation. At the 17k-step checkpoints:

Model	K=8 KB/tok	vs. Dense (K=8)
Dense d24	12,348	1.00×
ctrl	3,035	0.246×
λ=0.003	3,015	0.244×
λ=0.005	2,973	**0.241×**

Entropy consolidation at λ=0.005 adds a further 2% reduction in bytes-in-motion beyond the structural reduction of SR-Core itself (4.1× vs. dense).

6.8 Interpretation

What entropy minimization does: It makes the router's pre-topk distribution more concentrated, reducing the diversity of routing paths across tokens. This reduces the effective routing vocabulary (unique core combinations) and improves LRU cache hit rates.

What it does not do: It does not directly maximize cross-token routing overlap. The softfull ablation shows that cross-token similarity is a different degree of freedom. Entropy minimization acts within a single token's routing decision; temporal routing consistency is a side effect, not the direct target.

Where to operate:

- **λ=0.003** is the recommended balanced operating point — Pareto improvement in both cache and quality at s0/s1 (full Pareto, 2 seeds); quality consistency extended to 4 seeds (s0–s3), cache side not separately measured for s2/s3.
- **λ=0.005** is the preferred cache/systems operating point — largest consistent cache gain (−7.5% at K=24), quality effect seed-dependent (not a consistent cost or benefit).

- **λ=0.007** is the cache-best boundary — lowest absolute bytes/token in the sweep (K=24: 1,009 KB/tok), but the additional gain over λ=0.005 is only ~1.1% and quality reverts toward ctrl levels. Not the preferred operating point.

6.9 What This Chapter Does Not Show

- Wall-clock inference speedup *due to entropy consolidation* (the cache/locality improvement is measured in simulated bytes/token; whether it translates to additional throughput gain on top of the architectural win demonstrated in Chapter 5b.4 is untested)
- Dense-quality parity (dense d24 remains the upper bound)
- Scaling behavior beyond n=64 or this training corpus
- Downstream task performance

Data sources: `data/eval/entmin/eval_compare_*.json`, `data/eval/entmin/eval_quality_*.json`, `data/eval/entmin/offload_sim_17k.json`, `data/eval/entmin/eval_compare_softfull.json`

Chapter 7: Discussion and Future Work

7.1 What This Work Demonstrates

This monograph has investigated SR-Core, a recursive block-sparse language model architecture with a hard working-set guarantee (WS=k), and characterized the effect of entropy-based router consolidation on cache efficiency and language quality.

Demonstrated properties

Architectural:

- The WS=k guarantee holds by construction (Chapter 2) and empirically at all tested bank sizes n ∈ {16, 32, 64, 128, 256} on TinyStories (Chapter 3)
- Sparse routing (k=4 of n) outperforms dense execution (16/16) at equal compute on synthetic tasks; pure recursion without routing is strictly dominated
- Block specialization is reproducible (MI_norm = 0.197 ± 0.030 over 3 seeds)

Routing structure:

- Two-phase routing emerges in free-routing variants: exploration at r=1 ($J_1 \approx 0.20$), collapse at r≥2 ($J \approx 0.984$)
- Entropy-based consolidation induces a controllable axis: increasing λ monotonically reduces routing entropy, unique core combinations, and simulated bytes-in-motion
- Pre-selection entropy is the correct intervention point (softfull negative control moves in the wrong direction)

Cache efficiency:

- λ=0.003 is a Pareto improvement over unregularized baseline: cache efficiency and code generalization both improve, replicated across two independent seeds
- At K=8, SR-Core requires 4.1× fewer simulated bytes/token than dense layer-offloading

Quality:

- Dense d24 is the quality ceiling. Under one fixed protocol (Chapter 5a.5, seen contiguous loss), SR-Core b64 k8 R6 reaches ~5.30 (three seeds excl. a low-loss outlier) vs. Dense d24's 4.79 — ~0.5 nats seen (~0.34 held-out) — and Dense achieves this with half the block-applications per token (24 vs. 48). SR-Core does not win on language quality (Chapter 5a.5)
- The gap is not explained by parameter count or block-application compute in the tested regime: a param- *and* compute-matched SR-Core (8.75M params, 6.29M apps/token = Dense d24) still trails by ~0.48 nats seen / ~0.36 held-out (Chapter 5a.5). The evidence

points to the narrow-active-set + recursion format as the source. Whether the gap narrows at convergence or scale remains open (Section 7.5.6). (Both models are still descending at their stops, so the figure is a lower bound, not a converged value)
- This is by design, not a shortfall: SR-Core's contribution is transfer efficiency, not quality. Additional block compute is not the limiting term in the transfer-bound deployment regime — provided sparse dispatch is fused enough that per-block kernel-launch overhead is sub-dominant (achieved in Chapter 5b.4 via grouped matmul). The quality gap is the price paid for moving k=8 instead of 24 blocks per token
- λ=0.003 entropy consolidation does not degrade quality relative to ctrl

7.2 What This Work Does Not Demonstrate

Wall-clock inference speedup at scale. A real RAM→VRAM prototype (Chapter 5b.4) now demonstrates the wall-clock advantage *at small scale*: with a grouped block matmul (removing the per-block kernel-launch overhead that made a naive implementation launch-bound), SR-Core reaches 297 tok/s vs. 205 tok/s for dense layer-offloading at a VRAM budget below the dense model size — a 1.45× measured advantage on an RTX 2060. What remains undemonstrated is that this *grows* at deployment scale (large blocks, model $\gg$ VRAM, where the projection predicts a much larger gap), under asynchronous stream overlap, and in batched (not batch-1) serving. The mechanism is now measured; its magnitude at scale is still a projection.

Dense-quality parity. Entropy minimization shapes routing structure; it is not a path toward closing the quality gap with dense models.

Large-scale validation. All experiments use a ~19M parameter model on a ~6.6M token corpus. Whether the WS=k guarantee retains its cache-efficiency advantage at 7B or 70B parameters (where n $\gg$ 64) is untested.

Downstream task evaluation. All evaluation is on held-out language modeling loss (next-token cross-entropy). No downstream task (classification, generation quality, reasoning) has been tested.

Leiterbahn index. The routing trace analysis sufficient to build a block-prefetch index has not been implemented. The Leiterbahn (”conductor track”) concept — a prefetch index mapping routing entry signatures to block-load plans — remains a design direction, not an experimental result (see Section 7.5.3).

7.3 Relationship to the Original Research Plan

This project began with a broader 7-phase research plan; this monograph reports what was actually built and measured against it:

Phase	Plan	Status
1	Feasibility, tunnel emergence, anytime property	Complete
2	Harder routing (Gumbel, temperature curriculum)	Partial (top-k with noise)
3	3D spatial topology, trainable coordinates	Not implemented
4	Leiterbahn index	Not implemented
5	I/O loss, hardware-cost simulation	LRU simulation + real prototype (Ch. 5b.4)
6	Dynamic halting	Not implemented
7	Real RAM→VRAM streaming	Implemented at small scale (Ch. 5b.4); deploy

The work presented here covers Phase 1 (fully), a component of Phase 2 (entropy regularization as an alternative to temperature curriculum), and Phase 5 (LRU simulation plus a

first real-hardware RAM→VRAM prototype at small scale). The 3D topology and Leiterbahn concepts remain important directions but were not the focus of this experimental program.

The entropy regularization approach (Chapter 6) was not part of the original plan; it emerged from the observation that routing collapse at deep steps was a training artifact, and that the pre-topk entropy was the appropriate target for intervention.

7.4 The Most Important Open Question

The fundamental question was whether the transfer reduction demonstrated at simulation level (4.1×) survives real hardware measurement as a wall-clock **throughput** gain.

A real RAM→VRAM prototype (Chapter 5b.4) has now answered this in the affirmative at small scale. On the *transfer axis*: at the measured H2D bandwidth (~11 GB/s on an RTX 2060) SR-Core moves 6.7× fewer bytes/token than dense layer-offloading, and the simulated bytes/token reproduce exactly. On the *wall-clock axis*: a first naive implementation was kernel-launch-bound and slower than dense, but once the k per-block calls of each recursion step are fused into one grouped matmul, SR-Core reaches 297 vs. 205 tok/s — a 1.45× measured throughput advantage, with the dense baseline transfer-bound (it reloads all layers per token at a VRAM budget below its size) and SR-Core moving 6× less.

The chain *fewer bytes → more tokens/second* is therefore no longer only simulated; it is measured end-to-end on real hardware. The open question that remains is one of *magnitude*, not existence: does the advantage grow at deployment scale (large blocks, model ≫ VRAM, where the projection predicts far more than 1.45×), under asynchronous transfer/compute overlap, and in batched serving? The mechanism is demonstrated; its payoff at scale is the remaining work.

7.5 Future Directions

The open questions below are not presented as limitations to apologize for. Each is a discrete, well-scoped experiment that a group with the right hardware or the right curiosity can pick up where this program stopped.

Open question	Minimal next test
Quality gap convergence	Train Dense d24 and matched SR-Core to ~40k steps from 15k snapshots (~
Deployment-scale streaming	Measure wall-clock throughput with a block bank that genuinely exceeds VR
Batched serving	Measure union-of-routes at batch size > 1
Leiterbahn index	Record routing traces for 10k+ sequences; cluster by path; test prefetch hit
Attention blocks	Replace MLP blocks with attention-like blocks, retrain

7.5.1 Deployment-Scale Streaming

The RAM→VRAM prototype exists (Chapter 5b.4) and has cleared three milestones: the real transfer measurement; the grouped block matmul that removed per-block kernel-launch overhead (with which SR-Core surpasses dense layer-offloading, 1.45× at small scale); and asynchronous two-stream overlap (prefetch the next token's blocks during the current token's compute), which adds a further 5–9% in the transfer-heavy regime (small VRAM cache) and, as expected, nothing when the cache is large enough that transfer is already free. The end-to-end measured advantage on an RTX 2060 reaches ~1.6× with overlap. The remaining steps: (1) batched serving rather than batch-1 (the union-of-routes problem — the active block *union* across batch items widens the effective working set, potentially reducing the k/n advantage); (2) measurement at deployment scale (large blocks, model ≫ VRAM) where the projection predicts the advantage grows well beyond the small-scale figure, and where the compute

path is FLOP-bound rather than partly launch-bound, so overlap should hide a larger transfer fraction.

Hardware target: GPU with ≤8 GB VRAM, ≥64 GB CPU RAM, where the dense 7B+ model does not fit in VRAM but the k=8 active blocks of SR-Core do.

> **Continuation spec:** Measure wall-clock throughput at deployment scale (model whose total weight bank exceeds VRAM by 4–10×, large blocks). Separately, measure batch>1 throughput to quantify the union-of-routes effect. Expected outcome range: advantage grows substantially (transfer-dominated, FLOP-efficient path) or is eaten by batching (union collapses effective sparsity). Either result is informative.

7.5.2 Bank Size Scaling

The 4.1× transfer reduction at n=64 benefits from the fact that k/n = 8/64 = 12.5%. At n=512 with k=8, the fraction drops to 1.6% — a 63× theoretical reduction vs. dense. Whether SR-Core training remains stable at n=512 and whether the quality gap to dense remains manageable at that scale is a critical open question.

> **Continuation spec:** Train SR-Core at n=128 and n=512 on the same HeteroMini-v1 corpus with the same hyperparameters. Measure held-out loss and routing entropy collapse. If training is stable, measure offload bytes/token to confirm the theoretical k/n scaling.

7.5.3 3D Topology and Leiterbahn

If routing paths are reproducible (MI_norm > 0.197, routing structure replicates across seeds), then routing trace analysis on a large validation set should reveal structure suitable for Leiterbahn indexing. The original plan describes this step in detail. It requires:

1. Routing trace recording on 10,000+ validation sequences
2. Tunnel clustering by entry region and path similarity
3. Index construction mapping entry signatures to block prefetch plans
4. Prefetch-precision measurement on held-out sequences

> **Continuation spec:** Run routing trace collection on the existing 15k checkpoints (s0–s3) against the full HeteroMini-v1 validation set. Compute per-token block prediction accuracy from the index (does the correct block appear in the top-3 prefetch candidates?). If hit rate > 80%, the index is operationally useful; if it is near random, the assumption of reproducible routing structure at this scale is falsified.

7.5.4 Targeted Entropy Objectives

The current objective ($\lambda \cdot H(p)$) applies uniform pressure across all tokens. A targeted variant — penalizing only when H(p) exceeds a threshold H_target — concentrates pressure on high-entropy tokens. The target-entropy variants tested in Chapter 6 converge slowly in 2,000 steps but may be more effective at longer continuation. The interaction between target-entropy objectives and load-balancing terms (which encourage entropy at the batch level) is also unexplored.

> **Continuation spec:** Resume target-entropy models from their 2,000-step checkpoints to 15,000 steps. Compare cache efficiency and quality against the standard $\lambda \cdot H(p)$ baseline at matched training budget. The expected result is faster cache convergence; the risk is quality degradation on high-entropy tokens that currently carry semantic load.

7.5.5 Attention Blocks

MLP blocks have low arithmetic intensity (~0.25 FLOPs/Byte at d=256, h=512). Attention blocks have substantially higher intensity (~128 FLOPs/Byte at seq=512, d=256), making them more favorable for the RAM→VRAM scenario: more computation per loaded byte. Replacing MLP blocks with attention blocks while maintaining SR-Core routing would increase the transfer-efficiency of the architecture.

> **Continuation spec:** Replace the MLP block with a multi-head attention block, keep the routing mechanism unchanged. Measure (a) arithmetic intensity per loaded byte, (b) quality on HeteroMini-v1 at matched parameters, (c) wall-clock throughput under forced offloading. The hypothesis is that the throughput advantage grows because the GPU has more useful work to do per byte transferred; the counter-risk is that attention routing dynamics differ from MLP and routing collapse requires different regularization.

7.5.6 Convergence of the Quality Gap (Cheap, Decisive)

The ~0.5-nat quality gap (Chapter 5a.5) is measured at 15k steps, where both the param/compute-matched SR-Core and Dense d24 are still descending (~0.2 nats per 2,500 steps each). It is therefore a lower bound, not a converged value. Training both to convergence (~40k+ steps; ~6–8 h on the same RTX 2060, with auto-resume from the 15k snapshots — no new setup) would settle whether the gap narrows, holds, or widens at convergence. The descent rates are comparable at 15k, so this data shows no sign of SR-Core catching up — but a weight-tied recursive core may simply need more training to ”drill in” the shared refinement operator, and that is exactly what this run would test. It is the one open question in this work that the available hardware can answer cleanly; the rest (scale, downstream tasks, the Leiterbahn index) require resources and curiosity beyond this program.

> **Continuation spec:** Resume both models from their 15k checkpoints to 40k steps (auto-resume, no new setup, ~6–8 h on the same RTX 2060). Measure held-out loss every 2,500 steps. Three outcomes: (1) gap narrows → weight-tied recursion was under-trained, SR-Core may be competitive at longer horizon; (2) gap holds → the ~0.5-nat cost is stable, the current claim is confirmed; (3) gap widens → format under-performs as training length increases, a stronger negative result than the current lower bound.

7.6 Claim Boundaries

This monograph claims:

> SR-Core provides a hard working-set guarantee WS=k, and entropy-based router consolidation creates a reproducible cache/locality axis without degrading language quality at the Pareto-optimal operating point (λ=0.003, two seeds).

It also claims, now backed by measurement rather than simulation:

> Under RAM→VRAM offloading on consumer hardware (RTX 2060), SR-Core achieves a real wall-clock throughput advantage over dense layer-offloading (~1.6×) at small scale, at a measured quality cost of ~0.5 nats (param- and compute-matched, at this training horizon and scale; whether the gap changes at convergence or larger scale remains open).

It does **not** claim:

> dense-quality parity, a converged quality gap, or demonstrated efficiency *at deployment scale* (≫VRAM models, batched serving).

The honest framing is: the chain *fewer bytes → more tokens/second* is now demonstrated end-to-end on real hardware, and the quality price for it is measured and not attributable to parameter or compute budget at this training horizon. What remains open is magnitude at scale — whether the small-scale throughput win and the quality gap both move favourably as models grow — which requires hardware beyond this program.

7.7 Conclusion

Block-sparse recursive language models with shared routing (SR-Core) combine two properties that are jointly necessary for predictable parameter streaming: a hard active- set bound (WS=k) and, with entropy regularization, a controllable cache locality axis.

Both properties have been demonstrated at the scale studied, and the simulation-to-real translation has been shown at small scale (Chapter 5b.4: 297 vs. 205 tok/s, ~1.6× with overlap). The primary remaining task is a *deployment-scale* demonstration — batched serving and block sizes large enough that the transfer-bound advantage grows as projected on a model that genuinely exceeds VRAM — which requires hardware beyond this program.

Chapter 8: Related Work

SR-Core sits at the intersection of four lines of work: sparse mixture-of-experts routing, looped/recurrent depth, parameter offloading under memory constraints, and routing regularization. This chapter situates the contributions of this monograph against each.

8.1 Mixture of Experts

Sparse gating in transformer models routes tokens to subsets of expert feed-forward networks (Shazeer et al., 2017; Fedus et al., 2022). Standard MoE architectures activate a different expert subset at each layer, so a model with depth D and per-layer budget k touches up to D × k distinct parameter blocks per token — the transfer budget grows with depth.

SR-Core differs in two respects. First, it uses a **single shared block bank** applied recursively: there are no per-layer parameters, only one bank of n blocks reused R times. Second, the SR-Core constraint fixes the routing decision at r = 1 and reuses the same selection S_t for all subsequent steps r = 2 … R. The result is WS = k exactly, independent of R — depth and transfer budget are decoupled.

Load-balancing auxiliary losses (Lepikhin et al., 2021; Fedus et al., 2022) prevent expert collapse in standard MoE. SR-Core does not require load balancing as a prerequisite (the entropy objective addresses a different property — cross-token routing consistency, not within-batch utilization), but the two terms can be combined.

8.2 Looped and Recurrent Depth

The Universal Transformer (Dehghani et al., 2019) applies a single shared layer repeatedly, achieving depth without parameter growth — structurally the closest prior to SR-Core's recursive reuse. Deep Equilibrium Models (Bai et al., 2019) seek fixed points of a shared transformation, iterating until hidden state converges.

SR-Core is similar in using shared parameters across recursion steps but differs in intent and measurement. It does not seek convergence: state change at each step is empirically 12–22% of the hidden norm, and this persists through training (Section 5a). The goal is not a fixed point but a sequence of refinements within a bounded active set. More importantly, neither the Universal Transformer nor DEQ addresses **block-level prefetching or working-set control**: in both, the full shared layer is loaded at every step, so the transfer budget is identical to a dense model of the same depth. The WS = k guarantee is specific to SR-Core's top-k bank selection.

8.3 Parameter Offloading

FlexGen (Sheng et al., 2023) and related systems (Aminabadi et al., 2022; HuggingFace Accelerate) optimize layer-offloading schedules for dense transformers under memory constraints,

overlapping compute and transfer to maximize throughput. These approaches assume the **full parameter set must be accessed** — scheduling optimizes *when* each layer is transferred, not *whether*. For a model that does not fit in VRAM, every layer must cross the bus at some point per forward pass.

SR-Core reduces *what* needs to be loaded per token. The transfer budget per token is k blocks rather than all n (or equivalently, all D layers in a dense model). The two approaches are **complementary**: an SR-Core model could apply an optimized transfer schedule (FlexGen-style) on top of an already-reduced per-token budget. The streaming prototype in Chapter 5b.4 demonstrates the combined effect at small scale without a scheduler; adding a Leiterbahn-based prefetch plan (Chapter 7.5.3) would be the natural next integration.

8.4 Routing Regularization

Auxiliary objectives in MoE training include load-balancing losses (Fedus et al., 2022), z-loss for router stability (Zoph et al., 2022), and router noise regularization (Shazeer et al., 2017). These objectives target **different failure modes**: dead experts (utilization collapse), unstable softmax logits (training divergence), or inadequate exploration during early training.

The entropy objective introduced in Chapter 6 targets a different quantity entirely: **pre-topk concentration**, which determines how consistent the routing decision is across tokens. High pre-topk entropy means the router's probability mass is spread evenly before the top-k cut, producing random-looking selections that change token to token — good for coverage, bad for cache locality. Low entropy means the router's mass concentrates on a small preferred set, producing stable, reusable routing paths. This is orthogonal to load balance (a per-batch statistic over which blocks get used) and to training stability.

No prior routing regularizer is motivated by cross-token cache locality or by the working-set geometry of a RAM→VRAM streaming scenario.

8.5 Weight Sharing and Bank Reuse

SHA-RNN (Merity, 2019) and related weight-sharing recurrent architectures reduce parameter count through shared weight matrices. The reuse is **structural** — the same weights are always applied, with no content-dependent selection. SR-Core combines weight sharing (same blocks across recursion steps) with content-dependent top-k selection: the active subset per token is a function of input, while the total active count is fixed at k. This makes SR-Core's working set simultaneously predictable (always k blocks, never more) and adaptive (which k blocks depends on the token).

8.6 Summary of Positioning

Property	Dense	MoE	Universal Transformer	SR-Core
Working set per token	All params	k per layer	All (shared)	k total (WS=k)
Routing per step	—	Independent	—	Fixed at r=1, reused
Transfer budget	D × full	D × k	D × full	k (independent of R)
Depth without param growth	No	No	Yes	Yes
Cache locality control	—	Load balance	—	Entropy regularizatio
Offloading addressed	Scheduling	Scheduling	No	What to load

The WS=k guarantee is the property that makes all the others tractable: it bounds the transfer budget, enables the entropy objective to have a well-defined target (stable active-block

identity across tokens), and provides the architectural basis for the prefetch-planning ideas in Chapter 7.5.3.

References cited: Shazeer et al. (2017) "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"; Fedus et al. (2022) "Switch Transformers"; Lepikhin et al. (2021) "GShard"; Dehghani et al. (2019) "Universal Transformers"; Bai et al. (2019) "Deep Equilibrium Models"; Sheng et al. (2023) "FlexGen"; Aminabadi et al. (2022) "DeepSpeed Inference"; Zoph et al. (2022) "ST-MoE"; Merity (2019) "Single Headed Attention RNN".